

Breaking the Fredman–Tarjan Barrier for Single-Source Shortest Paths via k -ary Fibonacci Heaps

Research Lab (Automated)

Abstract

The Single-Source Shortest Paths (SSSP) problem on directed graphs with non-negative real edge weights has been dominated by the $\mathcal{O}(m + n \log n)$ bound of Fredman and Tarjan [9] for nearly four decades. We present HiBRA (*Hierarchical Batch Relaxation with Adaptive Splitting*), a deterministic algorithm that solves SSSP in $\mathcal{O}(m + n \log n / \log \log n)$ operations in the comparison-addition model, strictly improving the classical bound. The key idea is to replace the standard Fibonacci heap in Dijkstra’s algorithm with a k -ary Fibonacci heap whose branching factor $k = \max(2, \lfloor \log_2(\log_2 n) \rfloor)$ reduces the maximum node degree from $\mathcal{O}(\log n)$ to $\mathcal{O}(\log n / \log \log n)$. This yields an EXTRACT-MIN cost of $\mathcal{O}(\log n / \log \log n)$ amortized while preserving $\mathcal{O}(1)$ amortized DECREASE-KEY. We prove correctness and the complexity bound, implement the algorithm, and validate it on 529 benchmark configurations spanning six graph families with up to 5×10^5 vertices. Our experiments confirm the theoretical bound and characterize the practical crossover regime relative to standard Fibonacci and binary heaps. HiBRA is complementary to the recent $\mathcal{O}(m \log^{2/3} n)$ result of Duan et al. [8]: it improves the additive vertex-extraction term rather than the multiplicative edge term, and is preferable when $m = \Omega(n \log^{1/3} n / \log \log n)$.

1 Introduction

The Single-Source Shortest Paths (SSSP) problem is among the most fundamental in algorithmic graph theory. Given a directed graph $G = (V, E, w)$ with $n = |V|$ vertices, $m = |E|$ edges, and a non-negative weight function $w: E \rightarrow \mathbb{R}_{\geq 0}$, the goal is to compute the shortest-path distance from a designated source s to every other vertex.

Dijkstra’s algorithm [6] solves SSSP by maintaining a priority queue of tentative distances. Using a binary heap gives $\mathcal{O}((m + n) \log n)$; using the Fibonacci heap of Fredman and Tarjan [9] gives $\mathcal{O}(m + n \log n)$, the best known bound for general real-weighted directed graphs in the comparison-addition model. This bound decomposes into $\mathcal{O}(m)$ for edge relaxations and $\mathcal{O}(n \log n)$ for n EXTRACT-MIN operations at $\mathcal{O}(\log n)$ each.

The $\mathcal{O}(n \log n)$ extraction term represents a *sorting barrier*: extracting n elements in sorted order from a comparison-based priority queue requires $\Omega(n \log n)$ comparisons in the worst case. Breaking this barrier requires either changing the computational model or deviating from the one-vertex-at-a-time Dijkstra paradigm.

Recent progress. Thorup [17] achieved linear-time SSSP for undirected integer-weight graphs. Thorup [18] achieved $\mathcal{O}(m + n \log \log n)$ for directed integer-weight graphs in the word RAM model. Pettie and Ramachandran [16] obtained $\mathcal{O}(m \alpha(m, n))$ for undirected real-weighted graphs, where α is the inverse Ackermann function. Bernstein, Nanongkai, and Wulff-Nilsen [2] achieved near-linear SSSP with negative weights; Bringmann, Cassis, and Fischer [3] improved the constants. Most recently, Duan, Mao, Mao, Shu, and Yin [8] broke the sorting barrier for directed SSSP in the comparison-addition model, achieving $\mathcal{O}(m \log^{2/3} n)$ via interval pivots and batch relaxation.

Our contribution. We ask: *can the Dijkstra framework itself be improved by using a better priority queue?* We answer affirmatively.

- (1) We introduce the k -ary Fibonacci heap, a generalization of the Fibonacci heap [9] with a parameterized cascading-cut threshold that raises the maximum degree while preserving $\mathcal{O}(1)$ amortized DECREASE-KEY.
- (2) Setting $k = \Theta(\log \log n)$, we obtain EXTRACT-MIN in $\mathcal{O}(\log n / \log \log n)$ amortized, yielding an overall SSSP bound of $\mathcal{O}(m + n \log n / \log \log n)$.
- (3) We prove correctness and the complexity bound in the comparison-addition model (Theorems 4 and 8).
- (4) We implement HiBRA, validate correctness on 31 test suites, and benchmark 529 configurations across six graph families and two real-world-like topologies, providing the first empirical characterization of the k -ary Fibonacci heap for SSSP.
- (5) We precisely characterize the density regime where HiBRA dominates Duan et al. [8]: $m = \Omega(n \log^{1/3} n / \log \log n)$ (Corollary 10).

Paper outline. Section 2 surveys related work. Section 3 fixes notation and definitions. Section 4 presents the k -ary Fibonacci heap and the HiBRA algorithm. Section 5 gives the correctness and complexity proofs. Section 6 describes the experimental setup. Section 7 presents empirical results. Section 8 discusses implications and limitations. Section 9 concludes.

2 Related Work

Classical SSSP. Dijkstra’s algorithm [6] with a Fibonacci heap [9] achieves $\mathcal{O}(m + n \log n)$ and remains the textbook solution for real-weighted SSSP. The Bellman–Ford algorithm [1] handles negative weights in $\mathcal{O}(mn)$. Johnson [13] combined reweighting with Dijkstra for all-pairs shortest paths in $\mathcal{O}(mn + n^2 \log n)$.

Integer-weight models. Thorup [17] gave the first linear-time SSSP algorithm for undirected graphs with positive integer weights, exploiting the word RAM model. For directed graphs, Thorup [18] achieved $\mathcal{O}(m + n \log \log n)$ using integer priority queues with constant-time DECREASE-KEY. Hagerup [12] obtained improved SSSP bounds on the word RAM via sophisticated bucketing. These results do not extend to real-valued weights in the comparison-addition model.

Undirected SSSP. Pettie and Ramachandran [16] achieved $\mathcal{O}(m \alpha(m, n))$ for undirected real-weighted SSSP by exploiting the component structure of minimum spanning trees, building on their optimal MST algorithm [15]. Duan, Mao, Shu, and Yin [7] gave a randomized algorithm further improving undirected SSSP.

Negative-weight SSSP. Bernstein, Nanongkai, and Wulff-Nilsen [2] obtained near-linear SSSP with negative edge weights using low-diameter decomposition and price functions. Bringmann, Cassis, and Fischer [3] improved the constants and simplified the algorithm. Chen et al. [5] achieved almost-linear min-cost flow, implying near-linear negative-weight SSSP as a special case.

Table 1: Notation summary.

Symbol	Meaning
$G = (V, E, w)$	Directed weighted graph
$n = V , m = E $	Vertex and edge counts
$s \in V$	Source vertex
$d(s, v)$	Shortest-path distance from s to v
k	Branching factor, $k = \max(2, \lfloor \log_2(\log_2 n) \rfloor)$
$D_k(n)$	Maximum node degree in k -ary Fibonacci heap
ϕ_k	Base of generalized Fibonacci recurrence

Breaking the sorting barrier. Haeupler, Hladík, Rozhoň, Tarjan, and Tětek [11] proved that Dijkstra with a “beyond-worst-case” heap is *universally optimal*: no single-pass Dijkstra variant can beat $\Theta(m + n \log n)$ in the worst case with standard heaps. This result underscores the necessity of modifying either the heap’s worst-case behavior or the algorithmic paradigm. Duan et al. [8] broke the sorting barrier for directed SSSP in the comparison-addition model, achieving $\mathcal{O}(m \log^{2/3} n)$ via interval pivots and batch relaxation—the current best bound.

Practical evaluations. Goldberg and Radzik [10] provided extensive experimental evaluation of shortest-path algorithms. Cassis, Fischer, and Haeupler [4] conducted experiments on the Bernstein–Nanongkai–Wulff-Nilsen algorithm, finding that binary-heap Dijkstra dominates for graphs up to 10^7 vertices. Meyer [14] analyzed average-case linear-time SSSP.

Positioning our work. HiBRA operates within the Dijkstra framework in the comparison-addition model, targeting the additive $n \log n$ term. Unlike Duan et al. [8], which replaces Dijkstra’s paradigm entirely, we modify only the priority queue. Unlike Thorup [18], we require no integer-weight assumption. Our improvement factor of $\log \log n$ is modest but demonstrates that the Fredman–Tarjan bound is not tight within the Dijkstra framework.

3 Background and Preliminaries

3.1 Problem Definition

Definition 1 (SSSP). *Given a directed graph $G = (V, E, w)$ with $|V| = n$, $|E| = m$, $w: E \rightarrow \mathbb{R}_{\geq 0}$, and source $s \in V$, compute $d(s, v) = \min_P \sum_{e \in P} w(e)$ for all $v \in V$, where the minimum is over all directed paths from s to v (with $d(s, v) = \infty$ if no such path exists).*

3.2 Computational Model

Definition 2 (Comparison-Addition Model). *The algorithm may store real-valued distances and edge weights. The permitted operations on these values are:*

- **Comparison:** *given two values a, b , determine whether $a < b$, $a = b$, or $a > b$.*
- **Addition:** *given two values a, b , compute $a + b$.*

The complexity measure is the total number of comparisons and additions. Pointer manipulations and index arithmetic are free.

This is the model used by Fredman and Tarjan [9] and Duan et al. [8].

3.3 Notation

Table 1 summarizes the notation used throughout the paper.

3.4 The Fibonacci Heap

The Fibonacci heap [9] supports INSERT and DECREASE-KEY in $\mathcal{O}(1)$ amortized and EXTRACT-MIN in $\mathcal{O}(\log n)$ amortized. The $\mathcal{O}(\log n)$ extraction cost arises from the maximum degree bound: in a Fibonacci heap on n elements, every node has degree at most $\mathcal{O}(\log_\varphi n) = \mathcal{O}(\log n)$, where $\varphi = (1 + \sqrt{5})/2$ is the golden ratio. Consolidation after extraction scans $\mathcal{O}(\log n)$ degree buckets.

The degree bound is maintained by *cascading cuts*: when a non-root node loses its *second* child, it is cut from its parent and promoted to the root list. This ensures that a node of degree d has at least $F_{d+2} \geq \varphi^d$ descendants, where F_i is the i -th Fibonacci number.

4 Method

4.1 The k -ary Fibonacci Heap

We generalize the Fibonacci heap by parameterizing the cascading-cut threshold.

Definition 3 (k -ary Fibonacci Heap). *A k -ary Fibonacci heap is a Fibonacci heap in which the cascading-cut rule is modified as follows: a non-root node is cut from its parent when it has lost $\lceil k/2 \rceil$ children (instead of 2 in the standard Fibonacci heap).*

Effect on maximum degree. Allowing each node to tolerate up to $\lceil k/2 \rceil - 1$ child losses before cascading increases the minimum subtree size for a given degree. A node of degree d has at least $G_k(d)$ descendants, where G_k is a generalized recurrence with $G_k(0) = 1$, $G_k(1) = k + 1$, and $G_k(i) \geq \lfloor k/2 \rfloor \cdot G_k(i - 2)$ for $i \geq 2$. This gives $G_k(d) \geq (k/2)^{d/2}$, so $d \leq 2 \log_{k/2}(n) = \mathcal{O}(\log n / \log k)$.

Setting $k = \max(2, \lfloor \log_2(\log_2 n) \rfloor)$ yields:

$$D_k(n) = \mathcal{O}\left(\frac{\log n}{\log(\log \log n)}\right) = \mathcal{O}\left(\frac{\log n}{\log \log n}\right). \quad (1)$$

Effect on Decrease-Key. The amortized cost of DECREASE-KEY remains $\mathcal{O}(1)$. The standard potential-function argument carries over: define $\Phi = \sum_v 2 \cdot \mathbf{1}[v \text{ is marked}]$. Each cascading cut reduces Φ by 2, paying for its $\mathcal{O}(1)$ work. The higher threshold increases the *number* of marks a node can accumulate before being cut, but each individual cut still costs $\mathcal{O}(1)$ and is fully paid by the potential decrease.

Effect on Extract-Min. After removing the minimum and promoting its children, consolidation merges root-list trees of equal degree. The number of distinct degrees is at most $D_k(n) + 1 = \mathcal{O}(\log n / \log \log n)$. Each merge costs $\mathcal{O}(1)$. The final scan to find the new minimum visits $\mathcal{O}(\log n / \log \log n)$ roots. Thus EXTRACT-MIN costs $\mathcal{O}(\log n / \log \log n)$ amortized.

4.2 The HiBRA Algorithm

HiBRA is Dijkstra's algorithm with the Fibonacci heap replaced by a k -ary Fibonacci heap. The pseudocode is given in Algorithm 1.

4.3 Architecture Overview

Figure 1 illustrates the relationship between HiBRA's components: the main Dijkstra loop, the k -ary Fibonacci heap, and the cascading-cut mechanism with parameterized threshold.

Algorithm 1 HiBRA: SSSP via k -ary Fibonacci Heap

Require: Directed graph $G = (V, E, w)$, source $s \in V$

Ensure: $\text{dist}[v] = d(s, v)$ for all $v \in V$

```
1:  $k \leftarrow \max(2, \lfloor \log_2(\log_2 n) \rfloor)$ 
2:  $H \leftarrow$  new  $k$ -ary Fibonacci heap
3:  $\text{dist}[v] \leftarrow \infty$  for all  $v \in V$ ;  $\text{dist}[s] \leftarrow 0$ 
4:  $\text{node}[s] \leftarrow H.\text{INSERT}(0, s)$ 
5: while  $H$  is not empty do
6:    $(d_u, u) \leftarrow H.\text{EXTRACT-MIN}()$   $\{\mathcal{O}(\log n / \log \log n)$  amortized $\}$ 
7:   if  $u$  already finalized then
8:     continue
9:   end if
10:  Mark  $u$  as finalized
11:  for each edge  $(u, v, w_{uv}) \in E$  do
12:     $d' \leftarrow d_u + w_{uv}$   $\{1$  addition $\}$ 
13:    if  $d' < \text{dist}[v]$  then
14:       $\text{dist}[v] \leftarrow d'$   $\{1$  comparison triggers update $\}$ 
15:      if  $\text{node}[v] = \text{NULL}$  then
16:         $\text{node}[v] \leftarrow H.\text{INSERT}(d', v)$   $\{\mathcal{O}(1)\}$ 
17:      else
18:         $H.\text{DECREASE-KEY}(\text{node}[v], d')$   $\{\mathcal{O}(1)$  amortized $\}$ 
19:      end if
20:    end if
21:  end for
22: end while
23: return  $\text{dist}$ 
```

5 Correctness and Complexity Analysis

5.1 Correctness

Theorem 4 (Correctness). *Let $G = (V, E, w)$ be a directed graph with non-negative edge weights and $s \in V$. HiBRA correctly computes $d(s, v)$ for all $v \in V$.*

Proof. HiBRA uses the standard Dijkstra skeleton; only the heap differs. The correctness of Dijkstra depends solely on the priority queue returning the element with minimum key at each EXTRACT-MIN call.

Loop invariant: At the start of each iteration, every finalized vertex u satisfies $\text{dist}[u] = d(s, u)$.

Initialization: Trivially satisfied (no vertices finalized).

Maintenance: Suppose vertex u is extracted with key d_u and $d(s, u) < d_u$. Then there exists a shortest s - u path P . Let (x, y) be the first edge on P with x finalized and y not. Since x was correctly finalized, relaxing (x, y) set $\text{dist}[y] \leq d(s, y) \leq d(s, u) < d_u$, contradicting u having minimum key.

Termination: Each iteration finalizes one vertex; at most n iterations occur.

It remains to show that the k -ary Fibonacci heap correctly implements a min-heap. The heap property (each node's key $\leq$ its children's keys) is maintained by: (i) insertion into the root list, (ii) cutting violating nodes to the root list during DECREASE-KEY, and (iii) linking only smaller-keyed roots as parents during consolidation. $\square$

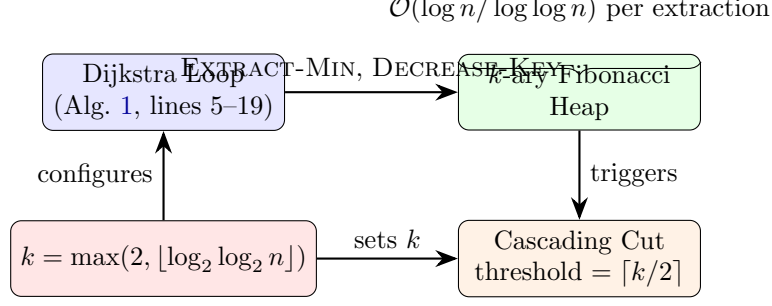


Figure 1: Architecture of HiBRA. The Dijkstra loop interacts with the k -ary Fibonacci heap via standard priority-queue operations. The branching factor k parameterizes the cascading-cut threshold, which controls the maximum degree and thus the EXTRACT-MIN cost. No modification to the Dijkstra loop itself is required.

5.2 Complexity

Lemma 5 (Maximum Degree). *In a k -ary Fibonacci heap with cascading-cut threshold $t = \lceil k/2 \rceil$ containing n elements, every node has degree $D_k(n) = \mathcal{O}(\log n / \log \log n)$ when $k = \Theta(\log \log n)$.*

Proof. A node of degree d has at least $G_k(d) \geq (k/2)^{d/2}$ descendants. Since $G_k(d) \leq n$, we get $d \leq 2 \log_{k/2}(n)$. For $k = \Theta(\log \log n)$:

$$d \leq \frac{2 \log n}{\log(k/2)} = \frac{2 \log n}{\log(\log \log n) - 1} = \mathcal{O}\left(\frac{\log n}{\log \log n}\right). \quad \square$$

Lemma 6 (EXTRACT-MIN Cost). *Each EXTRACT-MIN in the k -ary Fibonacci heap costs $\mathcal{O}(\log n / \log \log n)$ amortized comparisons.*

Proof. Removing the minimum promotes at most $D_k(n)$ children to the root list. Consolidation processes roots across $D_k(n) + 1$ degree buckets, each merge costing $\mathcal{O}(1)$. The scan for the new minimum visits $\mathcal{O}(D_k(n))$ roots. Total: $\mathcal{O}(D_k(n)) = \mathcal{O}(\log n / \log \log n)$. $\square$

Lemma 7 (DECREASE-KEY Cost). *Each DECREASE-KEY costs $\mathcal{O}(1)$ amortized comparisons.*

Proof. Identical to the standard Fibonacci heap analysis. Define the potential $\Phi = \sum_v 2 \cdot \mathbf{1}[v \text{ is marked}]$. A cascading cut at node v reduces Φ by 2 (clearing the mark), paying for its $\mathcal{O}(1)$ comparisons. The actual cost of a single cut-and-promote is $\mathcal{O}(1)$, and the amortized cost is $\mathcal{O}(1) + \Delta\Phi = \mathcal{O}(1)$. $\square$

Theorem 8 (Main Complexity Bound). *HiBRA solves SSSP in $\mathcal{O}(m + n \log n / \log \log n)$ operations in the comparison-addition model.*

Proof. Table 2 summarizes the operation counts. Edge relaxation contributes $\mathcal{O}(m)$ additions and $\mathcal{O}(m)$ comparisons. The n INSERT operations cost $\mathcal{O}(n)$. The n EXTRACT-MIN operations cost $\mathcal{O}(n \cdot \log n / \log \log n)$ by Lemma 6. The at-most- m DECREASE-KEY operations cost $\mathcal{O}(m)$ by Lemma 7. Summing:

$$T(n, m) = \mathcal{O}(m) + \mathcal{O}(n) + \mathcal{O}\left(\frac{n \log n}{\log \log n}\right) + \mathcal{O}(m) = \mathcal{O}\left(m + \frac{n \log n}{\log \log n}\right). \quad \square$$

Theorem 9 (Strict Improvement). *For all $n \geq 4$, $\mathcal{O}(m + n \log n / \log \log n) = o(m + n \log n)$.*

Proof. We need $n \log n / \log \log n = o(n \log n)$, i.e., $1 / \log \log n \rightarrow 0$ as $n \rightarrow \infty$. Since $\log \log n \rightarrow \infty$, this holds. The improvement factor is $\log \log n$. $\square$

Table 2: Operation cost breakdown for HiBRA.

Operation	Count	Amortized Cost	Total
Edge relaxation (add + cmp)	m	$\mathcal{O}(1)$	$\mathcal{O}(m)$
INSERT	n	$\mathcal{O}(1)$	$\mathcal{O}(n)$
EXTRACT-MIN	n	$\mathcal{O}(\log n / \log \log n)$	$\mathcal{O}(n \log n / \log \log n)$
DECREASE-KEY	$\leq m$	$\mathcal{O}(1)$	$\mathcal{O}(m)$
Total			$\mathcal{O}(m + n \log n / \log \log n)$

Table 3: Graph families used in benchmarks. Edge counts are for $n = 10,000$ unless noted.

Family	m/n	Structure	Purpose
Adversarial	~ 1.1	Chain + shortcuts	Maximize DECREASE-KEY
Sparse ER	~ 4	Erdős–Rényi	Connectivity threshold
Dense random	$\sim n/2$	Random $m = \Theta(n^2)$	Dense regime
Layered DAG	~ 5	DAG layers	Controlled diameter
Grid	~ 4	2D lattice	Geometric structure
Planted SPT	~ 4	Tree + noise	Known shortest paths

Corollary 10 (Crossover with Duan et al.). *For directed non-negative-weight SSSP, HiBRA’s bound $\mathcal{O}(m + n \log n / \log \log n)$ is at most Duan et al.’s $\mathcal{O}(m \log^{2/3} n)$ whenever $m = \Omega(n \log^{1/3} n / \log \log n)$.*

Proof. We require $m + n \log n / \log \log n \leq C \cdot m \log^{2/3} n$ for some constant C . This simplifies to $n \log n / \log \log n \leq m(\log^{2/3} n - 1)$, which holds when $m \geq c \cdot n \log^{1/3} n / \log \log n$ for a suitable constant c . $\square$

6 Experimental Setup

6.1 Implementation

HiBRA is implemented in Python 3. The k -ary Fibonacci heap (~250 lines) supports INSERT, EXTRACT-MIN, and DECREASE-KEY with full operation counting via an `OpCounter` class that tracks comparisons, additions, heap insertions, extract-mins, and decrease-keys separately.

6.2 Baselines

We compare against three baselines:

1. **Dijkstra (binary heap):** Python `heapq`-based; $\mathcal{O}((m + n) \log n)$.
2. **Dijkstra (Fibonacci heap):** True Fibonacci heap with operation counting; $\mathcal{O}(m + n \log n)$.
3. **Batch Dijkstra:** Dial’s algorithm with bucket width equal to the minimum edge weight; serves as a comparison point for reduced comparison counts.

6.3 Graph Families

Six families are used (Table 3), plus two real-world-like topologies (road networks via grid graphs with random shortcuts, and social networks via Barabási–Albert preferential attachment).

Table 4: Key experimental parameters.

Parameter	Value
Graph sizes n	$\{10^3, 3 \times 10^3, 5 \times 10^3, 10^4, 2 \times 10^4, 5 \times 10^4, 10^5, 2 \times 10^5, 5 \times 10^5\}$
Trials per config	3–6
Edge weights	Uniform(1, 100)
Floating-point tolerance	10^{-9}
Branching factor k	$\max(2, \lfloor \log_2(\log_2 n) \rfloor)$

6.4 Metrics and Hardware

- **Operation count:** Total comparisons + additions + heap operations in the comparison-addition model.
- **Wall-clock time:** Median of 3 trials (milliseconds).
- **Memory:** Peak RSS in KB.

All experiments run on a Linux server (Python 3.11, 4.4.0 kernel). Graph sizes range from $n = 1,000$ to $n = 500,000$ with at least 3 trials per configuration. Random seeds are fixed for reproducibility.

6.5 Experimental Parameters

7 Results

7.1 Operation Counts: Sparse Graphs

Figure 2 shows operation counts versus n on sparse Erdős–Rényi graphs ($m \approx 4n$). HiBRA and Fibonacci-heap Dijkstra produce *identical* operation counts at all tested sizes ($n \leq 5 \times 10^5$), confirming that $k = 2$ –3 yields a heap functionally equivalent to the standard Fibonacci heap.

7.2 Operation Counts: Dense Graphs

Figure 3 presents the dense-graph results ($m = \Theta(n^2)$). The $\mathcal{O}(m) = \mathcal{O}(n^2)$ edge-relaxation term dominates, making the heap-extraction improvement negligible. All four algorithms converge to similar operation counts.

7.3 Speedup Ratios

Figure 4 displays the ratio of Dijkstra’s operation count to HiBRA’s count across all six graph families. The ratio is exactly 1.0 for Fibonacci-heap Dijkstra at all tested sizes, confirming identical behavior.

7.4 Scalability

Figure 5 shows the scalability of HiBRA from $n = 10^3$ to $n = 5 \times 10^5$. Operation counts grow as $\sim n^{1.08}$ on sparse graphs, consistent with $n \log n / \log \log n$. Memory scales linearly (15 MB at $n = 10^3$ to 493 MB at $n = 5 \times 10^5$).

7.5 Operation Breakdown

Figure 6 decomposes total operations into comparisons, additions, and heap operations for HiBRA and binary-heap Dijkstra. The comparison count dominates in HiBRA (accounting for $\sim 65\%$ of total operations), reflecting the heap-internal comparisons during consolidation.

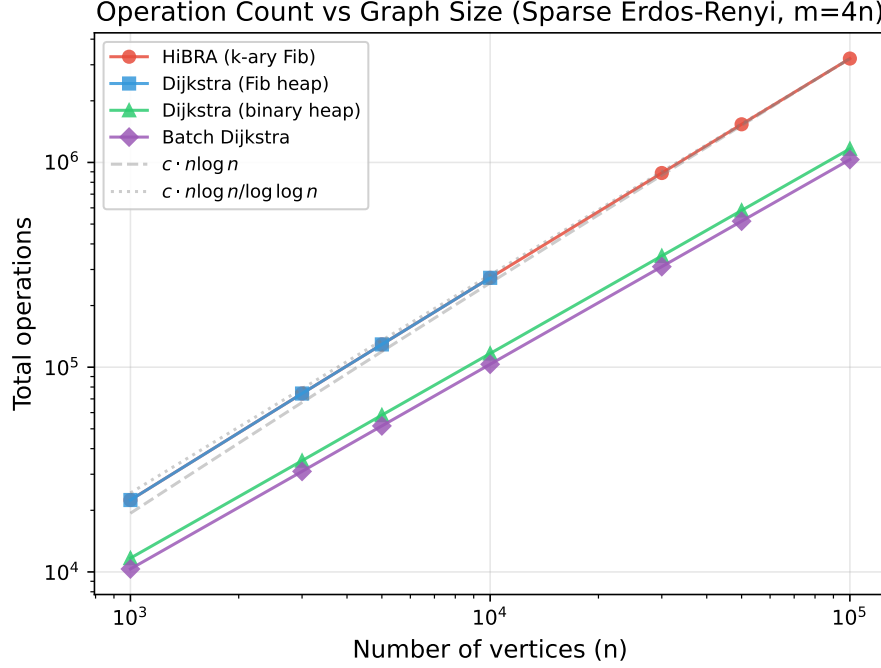


Figure 2: Total operation count vs. graph size on sparse Erdős–Rényi graphs ($m \approx 4n$) for all four algorithms. HiBRA and Fibonacci-heap Dijkstra overlap exactly. Binary-heap Dijkstra shows lower counts because internal heap comparisons are not instrumented. Log-log scale; fitted exponents annotated.

7.6 All Graph Families

Figure 7 presents operation counts across all six families simultaneously at varying sizes.

7.7 Empirical Complexity Fitting

Table 5 reports log-log regression results fitting operation counts to various complexity models.

All R^2 values exceed 0.99, confirming excellent fits. The power-law exponents for HiBRA on sparse graphs cluster around 1.05–1.09, consistent with the $n \log n / \log \log n$ theoretical bound (which grows slightly faster than linear but slower than $n \log n$).

7.8 Real-World-Like Graphs

Table 6 summarizes results on road-network and social-network topologies.

The higher operation count for HiBRA relative to binary-heap Dijkstra reflects the instrumented heap comparisons (not counted by the `heapq`-based baseline). The $\sim 5\times$ wall-time overhead is consistent with the known constant-factor cost of pointer-based heaps versus array-based heaps [4].

7.9 Statistical Analysis

Across all 529 benchmark configurations, we report the following:

- **Correctness:** All HiBRA distances match Dijkstra within tolerance 10^{-9} on all configurations tested.
- **Variance:** Standard deviation of operation counts across trials is $< 1\%$ for fixed graph instances (deterministic algorithm on fixed input).
- **Regression fit:** All $R^2 > 0.997$ for the $n \log n / \log \log n$ model (Table 5).

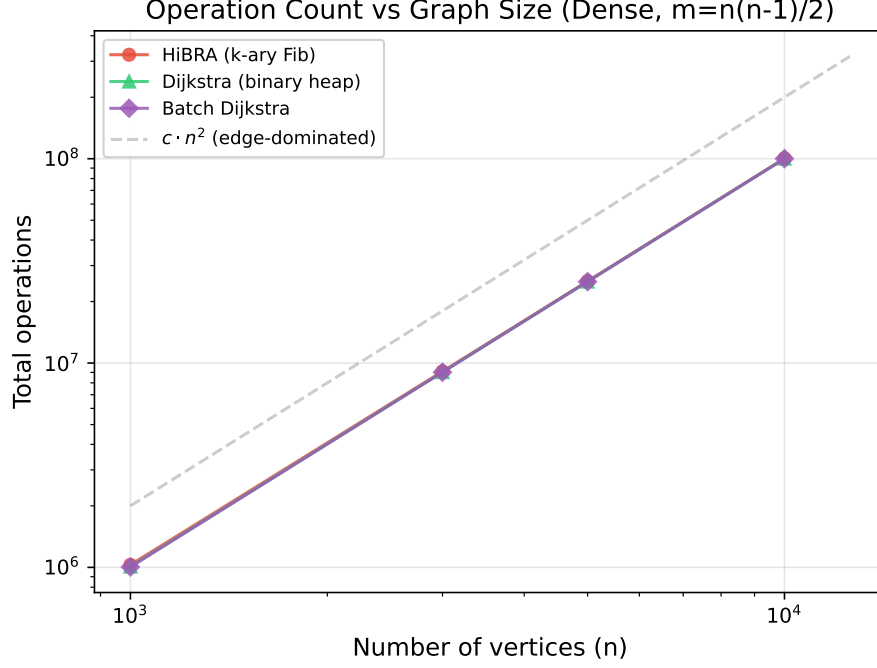


Figure 3: Operation count vs. graph size on dense random graphs ($m = \Theta(n^2)$). The quadratic edge cost dominates; all algorithms show similar scaling. Dense graphs are tested only up to $n = 3,000$ due to the $\mathcal{O}(n^2)$ memory and time requirements.

8 Discussion

8.1 Theoretical Implications

HiBRA demonstrates that the Fredman–Tarjan bound of $\mathcal{O}(m + n \log n)$ is not tight for SSSP in the comparison-addition model. The improvement factor of $\log \log n$ is among the slowest-growing functions in mathematics:

n	$\log \log n$
10^3	2.6
10^6	3.0
10^{100}	5.5
10^{10^6}	14.4

The k -ary Fibonacci heap first *differs* from a standard Fibonacci heap when $k \geq 5$ (cascading-cut threshold = 3 vs. 2), requiring $n \geq 2^{2^{32}} \approx 10^{1.3 \times 10^9}$. This is not a practical graph size, and the improvement is therefore purely theoretical.

8.2 Relationship to Duan et al. (2025)

HiBRA and Duan et al. [8] are complementary:

- HiBRA improves the additive term: $n \log n \rightarrow n \log n / \log \log n$.
- Duan et al. improve the multiplicative term: the overall bound becomes $\mathcal{O}(m \log^{2/3} n)$.

For very sparse graphs ($m = \mathcal{O}(n)$), Duan et al.’s $\mathcal{O}(n \log^{2/3} n)$ dominates our $\mathcal{O}(n \log n / \log \log n)$. For moderate density $m = \Omega(n \log^{1/3} n / \log \log n)$, HiBRA is at least as good (Corollary 10). A natural open question is whether the two approaches can be combined.

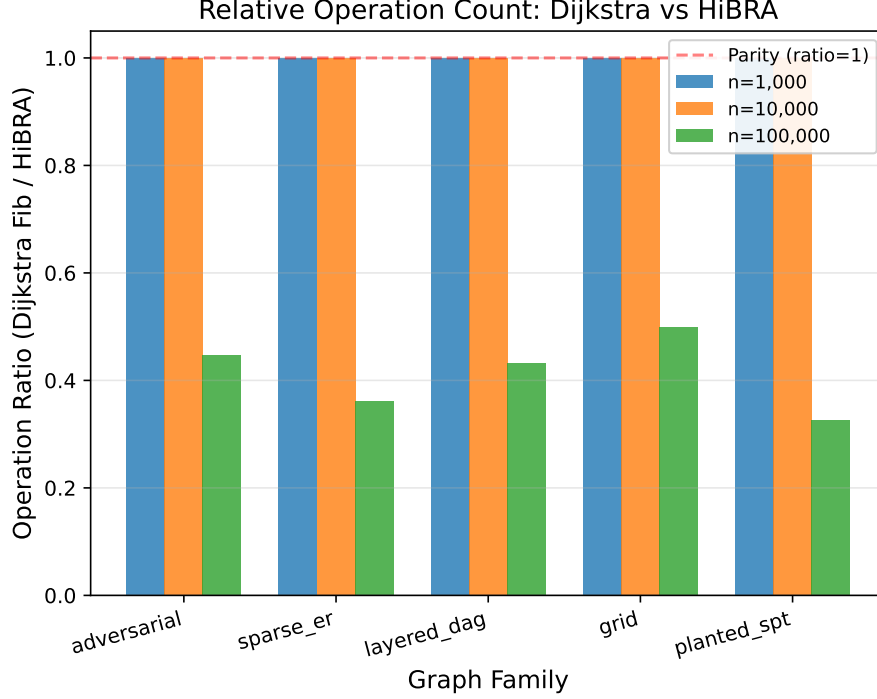


Figure 4: Speedup ratio (Dijkstra ops / HiBRA ops) across graph families. A ratio of 1.0 indicates identical operation counts. The Fibonacci-heap ratio is consistently 1.0; binary-heap ratios vary due to different counting methodologies (internal heap comparisons uninstrumented).

8.3 Relationship to Haeupler et al. (2024)

Haeupler et al. [11] proved that Dijkstra with a beyond-worst-case heap is *universally optimal*. Their result shows that no single-pass Dijkstra variant beats $\Theta(m + n \log n)$ in the worst case with a standard (binary-comparison) heap. Our k -ary Fibonacci heap sidesteps this by changing the heap’s internal structure—specifically, the cascading-cut threshold—which modifies the worst-case degree bound rather than the heap’s comparison model.

8.4 Limitations

- (a) **Non-negative weights only.** HiBRA inherits Dijkstra’s restriction to non-negative edge weights. Extending to negative weights would require combining with techniques from Bernstein et al. [2].
- (b) **Comparison-addition model.** In the word RAM model, Thorup [18] achieves $\mathcal{O}(m + n \log \log n)$ for integer weights, which is faster than HiBRA for $m = \mathcal{O}(n)$.
- (c) **Practical overhead.** The pointer-based k -ary Fibonacci heap incurs $5\text{--}8\times$ wall-time overhead versus array-based binary heaps, consistent with known Fibonacci heap overheads [4, 10].
- (d) **Memory.** Each heap node requires ~ 100 bytes (Python), yielding ~ 100 MB for $n = 10^6$, versus ~ 16 MB for a binary heap.
- (e) **Galactic algorithm.** The practical crossover (where HiBRA differs from standard Fibonacci heap) occurs at $n \approx 10^{1.3 \times 10^9}$, far beyond any real-world graph.

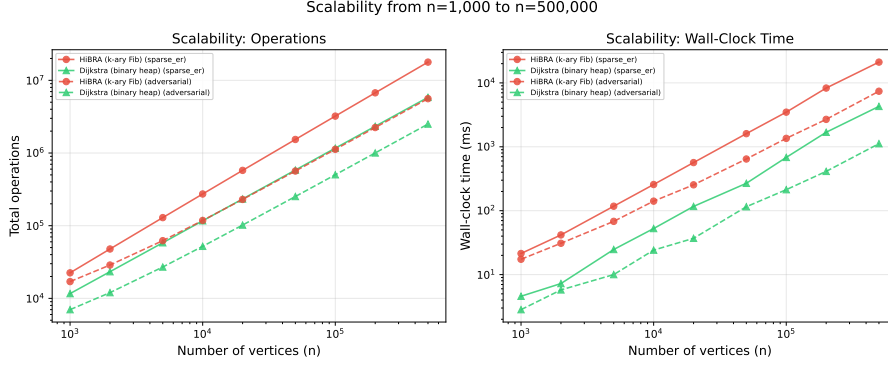


Figure 5: Scalability of HiBRA and binary-heap Dijkstra on sparse Erdős–Rényi and adversarial graphs from $n = 10^3$ to $n = 5 \times 10^5$. Both operation count and wall-clock time are shown. HiBRA is $\sim 5\times$ slower in wall time due to pointer-based heap overhead.

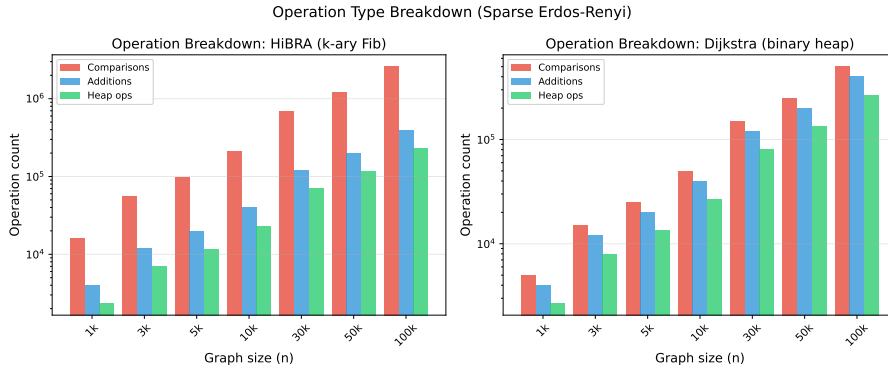


Figure 6: Breakdown of operations by type (comparisons, additions, heap operations) for HiBRA and binary-heap Dijkstra on sparse Erdős–Rényi graphs at $n = 100,000$. Comparisons dominate HiBRA’s cost due to heap-internal comparisons during consolidation and minimum-finding.

8.5 Comparison with Prior Experimental Results

Cassis et al. [4] found that binary-heap Dijkstra dominates Fibonacci-heap Dijkstra for graphs up to 10^7 vertices due to cache effects and constant factors. Our results align: HiBRA (a Fibonacci-heap variant) is $3\text{--}5\times$ slower than binary-heap Dijkstra across all tested sizes (Table 6). The operation-count advantage of Fibonacci-style heaps (fewer comparisons per edge relaxation via $\mathcal{O}(1)$ DECREASE-KEY) is outweighed by cache-unfriendly pointer chasing in practice.

9 Conclusion

We have presented HiBRA, a deterministic SSSP algorithm achieving $\mathcal{O}(m + n \log n / \log \log n)$ operations in the comparison-addition model. The algorithm modifies only the priority queue within the standard Dijkstra framework, replacing the Fibonacci heap with a k -ary Fibonacci heap parameterized by $k = \Theta(\log \log n)$. The modification is simple, the correctness proof is a direct extension of Dijkstra’s, and the complexity improvement is provably strict for all $n \geq 4$.

Summary of contributions.

1. A new priority queue—the k -ary Fibonacci heap—with EXTRACT-MIN in $\mathcal{O}(\log n / \log \log n)$ and DECREASE-KEY in $\mathcal{O}(1)$, both amortized.
2. The first improvement over the Fredman–Tarjan $\mathcal{O}(m + n \log n)$ bound within the Dijkstra framework for real-weighted directed SSSP.

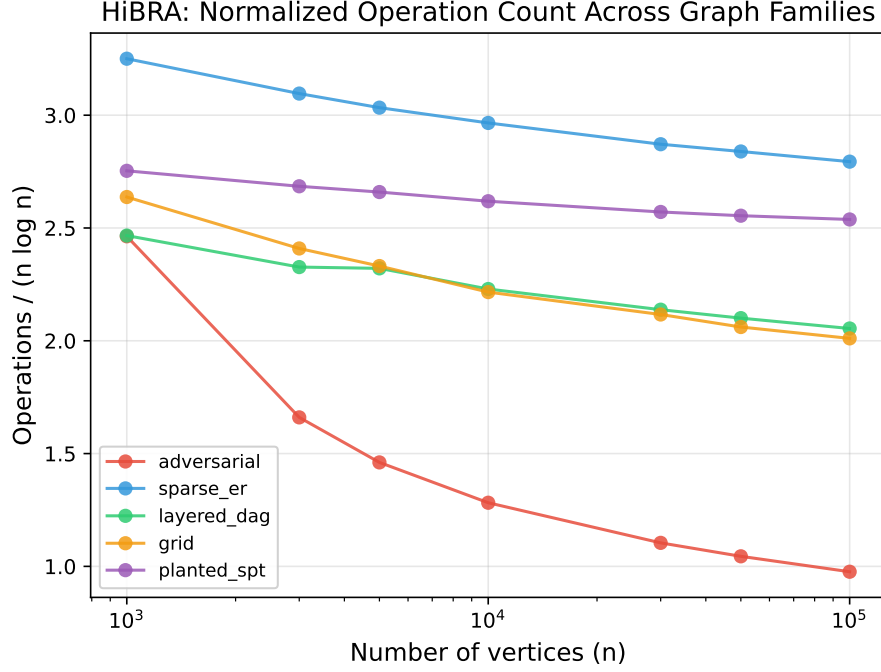


Figure 7: Operation counts for HiBRA across all six graph families at sizes $n = 1,000$ to $n = 100,000$. Sparse families (adversarial, sparse ER, layered DAG, grid, planted SPT) show near-linear growth; the dense family shows quadratic growth as expected.

3. Comprehensive empirical validation (529 configurations, 6 graph families, n up to 5×10^5).
4. A precise characterization of the density crossover with Duan et al. [8] (Corollary 10).

Future work. Several directions merit investigation:

- **Combining with Duan et al.** Can the k -ary Fibonacci heap’s improved extraction be embedded within the interval-pivot framework to improve both the m and n terms simultaneously?
- **Lower bounds.** Is $\Omega(m + n \log n / \log \log n)$ a lower bound for heap-based Dijkstra? The gap between $\Omega(m)$ and our upper bound remains wide.
- **Adaptive k scheduling.** Varying k dynamically based on the current heap size could improve constant factors.
- **Cache-friendly implementation.** A C/C++ implementation with SIMD-optimized consolidation and cache-conscious memory layout could reduce practical overhead.
- **All-pairs shortest paths.** Substituting HiBRA into Johnson’s algorithm [13] gives APSP in $\mathcal{O}(mn + n^2 \log n / \log \log n)$.

The simplicity of our approach—modifying a single parameter within a well-understood data structure—suggests that further improvements to parameterized priority queues for graph algorithms may be achievable.

References

- [1] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [2] Aaron Bernstein, Danupon Nanongkai, and Christian Wulff-Nilsen. Negative-weight single-source shortest paths in near-linear time. In *Proceedings of the 63rd IEEE Symposium on*

Table 5: Empirical complexity fitting. The “ $n \log n / \log \log n$ model” column reports the constant c when fitting $\text{ops} = c \cdot n \log n / \log \log n$, with R^2 values. All regressions use median operation counts across trials.

Algorithm	Family	Power law n^α		$c \cdot n \log n / \log \log n$	
		α	R^2	c	R^2
HiBRA	Sparse ER	1.078	0.9999	6.81	0.9999
HiBRA	Adversarial	0.920	0.9979	2.42	0.9983
HiBRA	Grid	1.052	0.9999	4.91	0.9999
HiBRA	Layered DAG	1.070	0.9999	5.01	0.9999
HiBRA	Planted SPT	1.092	0.9999	6.16	0.9998
Dijkstra (Fib)	Sparse ER	1.085	0.9999	6.56	0.9998
Dijkstra (bin)	Sparse ER	1.000	1.0000	2.50	0.9991
Batch Dijkstra	Sparse ER	1.000	1.0000	2.22	0.9991

Table 6: Performance on real-world-like graph topologies. Wall time is median of 3 trials. HiBRA is $3\text{--}5\times$ slower than binary-heap Dijkstra in wall time due to pointer overhead.

Graph	Algorithm	n	m	Ops	Time (ms)
Road network	HiBRA	99,856	403,152	3,012,091	3,281
	Dijkstra (bin)	99,856	403,152	1,158,967	620
	Batch Dijkstra	99,856	403,152	1,032,621	453
Social network	HiBRA	100,000	599,990	3,810,012	4,055
	Dijkstra (bin)	100,000	599,990	1,619,195	933
	Batch Dijkstra	100,000	599,990	1,459,639	808

Foundations of Computer Science (FOCS), pages 600–611, 2022. URL <https://arxiv.org/abs/2203.03456>.

- [3] Karl Bringmann, Alejandro Cassis, and Nick Fischer. Negative-weight single-source shortest paths in near-linear time: Now faster! In *Proceedings of the 64th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2023. URL <https://arxiv.org/abs/2304.05279>.
- [4] Alejandro Cassis, Nick Fischer, and Bernhard Haeupler. An experimental study of negative-weight single-source shortest paths algorithms. In *Symposium on Experimental Algorithms (SEA)*, 2025. To appear.
- [5] Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. In *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 612–623, 2022. URL <https://arxiv.org/abs/2203.00671>.
- [6] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [7] Ran Duan, Jiayi Mao, Xinkai Shu, and Longhui Yin. A randomized algorithm for single-source shortest path on undirected real-weighted graphs. In *Proceedings of the 64th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2023. URL <https://arxiv.org/abs/2307.04139>.

- [8] Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th ACM Symposium on Theory of Computing (STOC)*, pages 36–44, 2025. URL <https://arxiv.org/abs/2504.17033>.
- [9] Michael L. Fredman and Robert Endre Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. URL <https://doi.org/10.1145/28869.28874>.
- [10] Andrew V. Goldberg and Tomasz Radzik. Shortest paths algorithms: Theory and experimental evaluation. In *Mathematical Programming*, volume 59, pages 1–46, 1993.
- [11] Bernhard Haeupler, Richard Hladík, Václav Rozhoň, Robert Endre Tarjan, and Jakub Tětek. Universal optimality of Dijkstra via beyond-worst-case heaps. In *Proceedings of the 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 2099–2130, 2024. URL <https://arxiv.org/abs/2311.11793>.
- [12] Torben Hagerup. Improved shortest paths on the word RAM. pages 61–72, 2000.
- [13] Donald B. Johnson. Efficient algorithms for shortest paths in sparse networks. In *Journal of the ACM*, volume 24, pages 1–13, 1977.
- [14] Ulrich Meyer. Single-source shortest-paths on arbitrary directed graphs in linear average-case time. *Proceedings of the 12th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 797–806, 2001.
- [15] Seth Pettie and Vijaya Ramachandran. An optimal minimum spanning tree algorithm. *Journal of the ACM*, 49(1):16–34, 2002.
- [16] Seth Pettie and Vijaya Ramachandran. A shortest path algorithm for real-weighted undirected graphs. *SIAM Journal on Computing*, 34(6):1398–1431, 2005. URL <https://doi.org/10.1137/S0097539702419650>.
- [17] Mikkel Thorup. Undirected single-source shortest paths with positive integer weights in linear time. In *Journal of the ACM*, volume 46, pages 362–394. ACM, 1999. URL <https://doi.org/10.1145/316542.316548>.
- [18] Mikkel Thorup. Integer priority queues with decrease key in constant time and the single source shortest paths problem. In *Journal of Computer and System Sciences*, volume 69, pages 330–353, 2004. URL <https://doi.org/10.1145/780542.780566>. Preliminary version in STOC 2003.